Proposed revisions to the randomized DRL

What in the proposal is **not concrete**, which of its formulas and goals are **false as written**, and the **replacement expressions** - each one accompanied by the measurement that supports it. Equation numbers refer to *Randomized DRL with Multi-Resource (CPU, RAM, Disk) Capacity*; experiment numbers to this repository.

The proposal already contains a complete and correct ***mechanism***: the polytope D_{vec} (8), the latency gradient (19), the projected OGD step (20)-(25), the randomized rounding with a multi-resource repair (Sec. 3.4), the randomized threshold f(θ) (R3) and the resource metrics (30)-(34). What is missing is the **coupling** between the three components, the **precise form of the goals**, and the **cost of the algorithm**.

Five things have to change. (1) The download probabilities Q_t are introduced in (R1) and never used again - the algorithm must actually control *when* to download, not only *what* to keep. (2) Assumption 1 makes the gradient exogenous, so the state is not consistent with the dynamics it controls; the gradient has to be written in expectation over the algorithm's own download decisions. (3) Goal G3 compares a *realized* quantity with a *surrogate* comparator and is therefore false (3.7-9.6× the bound, growing with T); it must be split into a surrogate part and a realized part. (4) Goal G4 ("below the deterministic γ") and G5 ("report Λ_ϕ") are not mathematical statements and must be replaced by bounds. (5) The per-request cost (25-70× the deterministic one) must be brought down, which needs a cheaper projection and a cheaper rounding.

1. What the proposal already fixes (and what is therefore not to be changed)

element	formula	status in this repository
the feasible fractional cache	$D_{vec} = \{P : \sum a_{n,r}P_n \le 1\ \forall r,\ \sum P_n \le C\}$ (8)	implemented exactly (**sim/rand_drl.py**); the projection is verified against a cvxpy QP to 2×10^{-9} (exp07)
the latency surrogate gradient	$\text{grad}_{p_{n,t}} = -(1-q)l - q\cdot\min(l, \Delta m)$ (19)	implemented (**_ogd**); the variant (19′) proposed below makes q an *endogenous* quantity
the OGD step and the projection	$P_t = \Pi_{D_{vec}}(P_{t-1} - \eta \text{gradF}_t)$ (20)-(25)	implemented; the dual-ascent projection with warm start and the final scaling (25) is the one used
randomized rounding with repair	$s_k = 1[U_k < p]$, repair by increasing $p_n/\max_r a_{n,r}$	implemented; measured 40 % raw violations -> 0 % after the repair (exp08)
the randomized trigger	$f(\theta) = e^{\theta/M}/(M(e-1))$ (R3)	implemented; the rent-or-buy constant is exactly e/(e-1) = 1.582 (exp06)
the resource metrics	$U_{r,t}$, U^max, I_t, W_t (30)-(33)	implemented; they are what exposes the unsafe reductions (exp11)

2. The parts that are not concrete, and the revised expressions

Each item states what the proposal says, what is missing or wrong, and the replacement that is implemented and measured in this repository.

N1 - The download probabilities Q_t are never used

Proposal: (R1) announces "Marginal caching probabilities $P_t \in D_{vec}$ and download probabilities Q_t, with P_t updated by projected online gradient descent on a latency surrogate". But the gradient (19) contains the *exogenous* $q_{n^*,t}$, and the only decision variable for a download is the threshold θ of R3. Q_t appears in no equation, and the download decision is therefore outside the optimization: the algorithm cannot trade a download against the latency it avoids.

Revision (R1′ - two timescales). Split the state into the cache state P_t and the download state $Q_t = (Q_n)_n$, and let the download decision be a probability, not a threshold:

$$u_{n,t} = 1[\,U_{n,t} < Q_{n,t}\,],\quad Q_{n,t} = \text{clip}(\,Q_{n,t-1} - \eta_Q \cdot g^{(Q)}{}_t(n),\ 0,\ 1\,)$$

where $g^{(Q)}{}_t(n)$ is the marginal cost of one extra download, i.e. the download time M_n (or the residual budget per remaining request, B_{max}/requests_left, when a budget is imposed). The two time-scales are then: **the slow scale** updates P_t with the latency gradient of Sec. 3, **the fast scale** updates Q with the cost gradient, and the trigger threshold of R3 is replaced by the Bernoulli draw u above. This makes (G2) a first-class constraint instead of an afterthought: choose η_Q so that $E[\Sigma_{n,t}\, u_{n,t}M_n] \le B_{max}$ by construction.

Status: partially implemented - the budget-aware trigger already realizes the idea (the trigger probability is lowered to budget_left/(M_n·requests_left), exp08/exp15), but the *latency*-driven part of the Q update is not yet in the simulator. This is the single most valuable addition to the algorithm: it is what lets the randomized policy decide *whether* a download is worth its M_n instead of relying on the ski-rental threshold.

N2 - Assumption 1 makes the gradient exogenous

Proposal: Assumption 1 states that $q_{n,t}$ and $\Delta m_{n,t}$ are exogenous in F_t and that the cache and download indicators are conditionally independent given (P_t, Q_t). But q is exactly the indicator of the algorithm's own download decision, so the OGD state is updated with a gradient that does not correspond to the transition it is about to cause.

Revision (19′) - expectation over the algorithm's own randomness. Replace the realized q by its expectation, which the algorithm controls:

$\text{grad}_{p_{n,t}} = -[\,(1 - Q_{n,t})\cdot l_n + Q_{n,t}\cdot \min(l_n,$ the expected remaining download time$)\,]$, the expected remaining download time = E[remaining download time | a download of n was started at t - M_n]

and treat the difference between the realized and the expected gradient as noise. The expected-descent lemma then gives, with σ^2 the variance of the per-request gradient,

$\text{Reg}_T \le \sqrt{N}\cdot l_{max}\cdot\sqrt{T} + O(\sigma\sqrt{T})$ + (download-bias term, see N3)

The honest consequence is a **two-part statement**: the OGD part is exactly the Proposition-1 bound, and the stochastic part is the extra $O(\sigma\sqrt{T})$ term coming from the Bernoulli draw of the download decision. Assumption 1 can then be replaced by a **consistency condition** that is checkable in the simulator: $E[q_{n,t}]$ should be close to $Q_{n,t}$ (the realized in-flight rate versus the intended one), and the variance σ can be measured directly.

Status: the simulator already runs the *endogenous* variant (the realized q and Δm of the run are fed back into the gradient) and reports the regret of both: the endogenous regret stays below the bound in every configuration (exp09), and it is *negative* (the OGD state beats every fixed fractional cache) whenever the popularity shifts.

N3 - Goal G3 mixes a realized quantity with a surrogate comparator

Proposal (11): $E[\Sigma_t\, d_t(n^*_t)] - \min_{P\in D_{vec}} \Sigma_t\, F_t(P) \le \text{Reg}_T = o(T)$. The first term is the *realized* latency; the second is the *surrogate* loss of a fractional cache, which is allowed to cache a service **instantly**. The two terms are not comparable: the realized latency contains the download delays that the comparator never pays.

Measured (exp09): the literal regret is 3 912 / 5 119 / 6 592 / 9 595 / 12 917 for T = 500 ... 8 000, i.e. 3.7-9.6× the Proposition-1 bound and **growing with T** - so (11) is false as written, not merely loose.

Revision. Split the goal into three statements, each of which is checkable:

(a) **Surrogate regret** (unchanged, true): $\Sigma_t\, F_t(P_t) - \min_P \Sigma_t\, F_t(P) \le \sqrt{N}\, l_{max}\, \sqrt{T}$.
(b) **Download-charged inequality** (new, true): $E[\Sigma_t\, d_t] - \min_P \Sigma_t\, F_t(P) \le \text{Reg}_{Prop.1}(T) + D_T$,
where D_T is the download cost that *any* policy must pay on the same trace (the deterministic algorithm's cost is a valid proxy, since $E[\Sigma_t\, d_t] \le \Sigma_t\, F_t + D_T$ is exactly the statement that the surrogate under-counts the delay by at most the download cost).
(c) **Realized regret G3′** (new, the meaningful one): the realized latency of the randomized policy versus the realized latency of the **best static cache** played with the *same* download dynamics:

$\text{Reg}_{real}(T) = L_{rand}(T) - L_{static}(T)$, and the goal becomes $\text{Reg}_{real}(T) = o(T)$, i.e. $\text{Reg}_{real}(T)/T \to 0$

Verified in exp18 (b): the download-charged inequality holds at every horizon - the literal regret is 3 978 / 6 102 / 9 709 / 16 348 / 26 627 for T = 500 ... 8 000, while $\text{Reg}_{Prop.1}$ + D is 10 783 / 22 728 / 43 352 / 81 963 / 161 017. The gap is thus explained by the download cost that the instantaneous comparator does not pay.

And (c) is *not* satisfied as stated - the most useful finding of this section. Measured against the **offline-optimal** static cache (the support of the LP solution, played with the same download dynamics) the realized regret is 1 793 / 3 597 / 6 830 / 14 232 / 24 442 ms for T = 500 ... 8 000: a **constant 3.1-3.6 ms per request**, i.e. 25-34 % of the randomized policy's own latency. A constant per-request gap is a Θ(T) regret, **not** o(T), and the static set is also far cheaper (1 553-2 064 against 5 808-27 203) because it downloads only 7-9 services. On a *stationary* trace an offline-optimal fixed set dominates any online policy - so G3c has to be repaired a second time:

G3c′: the comparator must itself be **online** (the deterministic Online-DRL, or the best fixed fractional cache played online), and the goal must be stated as a **stationarity-relative** quantity - the adaptive policy pays an o(1) price per request *relative to the best online static policy* and gains (negative regret) exactly when the popularity shifts, which is what the surrogate measurements show (regret -659 / -840 / -898 / -1 166 / -1 486 ms for T = 500 ... 8 000 on a popularity-shifting sequence, exp09).

Concretely: (11) should be replaced by G3a (surrogate, provable), G3b (download-charged, provable, verified here) and G3c′ (realized, against an *online* comparator, meaningful only under a non-stationarity assumption). As written, the goal mixes three different quantities and cannot be satisfied by any algorithm.

N4 - The step size needs the horizon and the largest latency

Proposal (Prop. 1): $\eta = \sqrt{N}/(l_{max}\sqrt{T})$. An online algorithm does not know T, and l_{max} is a property of the catalogue that may be unknown or may change.

Revision (R-B). Use the standard decaying step size

$$\eta_t = 1/(l_{max}\cdot\sqrt{t}) \text{ , } \int_{t=1..T} l_{max}\cdot\eta_t \le 2\, l_{max}\sqrt{T} \Rightarrow Reg_T \le 2\sqrt{N}\, l_{max}\sqrt{T}$$

i.e. the same order with a constant 2 instead of 1, and **no horizon and no l_{max} needed a priori** (l_{max} can be replaced by a running maximum). **Measured (exp18):** the horizon-free schedule is not worse - it is *better*: the surrogate regret is 696 / 1 099 / 1 383 / 1 728 / 2 548 for T = 500 / 1 000 / 2 000 / 4 000 / 8 000 against 815 / 1 305 / 1 882 / 2 845 / 3 947 for the horizon-dependent step, always far below the bound ($2\sqrt{N}\, l_{max}\sqrt{T}$ = 5 940 ... 23 759). The revision is implemented as **eta_mode="adaptive"** and as **eta_sched="1/sqrt(t)"** in **sim/regret.py**.

N5 - The serving path k* is ambiguous

Proposal (R2): “the serving path is k* ~ Uniform{1,...,K}”. Read literally (a fresh k* at every time step) the served set changes at every request, which destroys the cache; read as “one path for the whole run” it is the sticky variant. The proposal does not say.

Revision (R-D). The serving path must be **sticky over a miss epoch** and redrawn only when the cache is reset (a new download completes, or the service is evicted). The churn is then bounded by the coupling the proposal asks for, $E[\#\text{changes}] \le \Sigma_t\, E||P_t - P_{t-1}||_1$, because the K paths share the persistent uniform vectors.

Measured (exp18, 5 seeds, 8 000 requests): the ambiguity is not cosmetic - on the same trace the sticky path gives latency 62 928 ± 7 114, cost 20 566 and 4 991 hits, while redrawing k* on every miss gives 75 498 (+20 %), 44 172 (2.1×) and 4 371 hits, and redrawing it at every request gives 77 019 (+22 %), 45 005 (2.2×) and 4 296 hits, with 8 000 redraws and 94 dropped services against 21 for the sticky path. The literal reading of the proposal loses 14 % of the hits and doubles the download cost. This is the quantitative justification for assumption A3, and it is implemented as **kstar_policy** in **sim/rand_drl.py**.

N6 - Goals G4 and G5 are not mathematical statements

G4 (12): “$\gamma_R = \max_I E[ALG_R(I)]/OPT(I)$ below the deterministic γ”. “Below” is not a bound. **Revision:** state it in the regime where it is provable and measurable, e.g. for the adversarial family of Theorem 1: $\gamma_R = O(1)$ against $\gamma_{det} = 2k$; and in general $E[ALG_R(I)] \le OPT(I) + O(\sqrt{T})$ with high probability, for an oblivious adversary. **Measured:** $\gamma_R \approx 3 \pm 1$ for k = 3 ... 24 while $\gamma_{det} = 2.00k$ ($R^2 = 1.000$) - the two models are not comparable as they stand, because a 2k ratio against an adversary is not the same experiment as an expected ratio against the optimum.

G5 (13): “if a slot reduction is used, its capacity loss Λ_ϕ is reported”. Reporting is not a guarantee. **Revision:** make it a decision rule with a proof obligation:

Safety (proved in the proposal). ϕ is a safe reduction iff $\phi(a) \ge \max_r a_r$ for every a, i.e. the weight must dominate the dominant resource. Hence $\Delta_\phi \ge 1$, and $\Delta_\phi = 1$ for the dominant-resource rule.
Decision rule (new). Use the slot formulation only if $\Lambda_\phi \le 1 + \varepsilon$ for a tolerance ε (e.g. 0.05); otherwise keep the vector formulation, whose per-step cost is O(N) instead of O(N log N) for the exact breakpoint sort.
New safe reduction with Λ close to 1 (new). $\phi_\beta(a) = \max_r a_r + \beta\cdot$(second largest component) is safe for any $\beta \ge 0$ and has $\Lambda_\beta \le 1 + \beta$, so it interpolates between the exact vector formulation ($\beta = 0$) and the dominant-resource slots ($\beta = \infty$).

Measured (exp11, exp12): the unsafe reductions are exactly those whose weights live on the simplex (entropy, AHP) or are uniform shadow prices: $\Lambda_\phi < 1$, a *lower* latency, and violations of the true resource limits in 55-85 % of the steps (up to 2.37× the capacity). The safe ones never violate but waste capacity (linear $\alpha = 1$:

Λ = 2.52, +71 % latency). A rule that simply says “report Λ” would have accepted the unsafe ones.

N7 - The per-request cost is 25-70× the deterministic one

Problem. Every request performs an O(N) Euclidean projection (dual ascent on 4 multipliers), an O(K·N) rounding of K paths and an O(N) repair of every violating path: 440-1 665 µs against 13-26 µs (exp14), i.e. ≈ 35 ms of CPU per second at 1 000 requests/s.

Revision. Four reductions, in order of expected payoff:

#	change	expected effect
1	**Active-set projection.** Only the services with $p_n > 0$ can be violating; project the support (|supp p| ■ N) and keep the rest at 0	O(|supp|) instead of O(N) per step; on Zipf traffic |supp| is a small fraction of N
2	**Amortize the rounding.** The coupling only requires that the sampled path changes by O(■p_t - p_{t-1}■$_1$) in expectation; resample *one* path per request (round-robin over the K paths) instead of all K	O(N) instead of O(K·N) per request, with the same marginals E[s] = p
3	**Lazy repair.** Repair a path only when it is actually served, not every path at every step	the repair cost is paid once per served path instead of K times
4	**Certified approximate projection.** Stop the dual ascent at a tolerance ε and apply the scaling (25): the result is exactly feasible and ε-optimal, with O(log(1/ε)) fewer iterations	fewer iterations of the O(N) dual ascent

With 1 and 2 the per-request cost should fall from O(N + KN) to O(|supp| + N), which the measurements suggest is a factor 5-10; the target is to be within a small constant factor of the deterministic Landlord. This is a concrete piece of engineering work, not a change of the mathematics, and it is the only way the randomized algorithm becomes deployable.

3. The revised algorithm

Randomized DRL v2 (the changes with respect to the proposal are marked):

step	action	formula / source
0	draw the K persistent uniform vectors $U_k \in [0,1]^N$; pick the serving path k* **sticky over a miss epoch** (N5)	proposal Sec. 3.4 + revision R-D
1	on a request for n*: update the **download probability** Q (N1) and the **cache state** P with the expected gradient (N2)	$z_{n,t} = p_{n,t-1} - \eta\cdot(1 - Q_n)l_n$ ... ; revision of (19)-(20)
2	project onto D_{vec} with the warm-started dual ascent and the final scaling	(21)-(25), unchanged
3	round: the served path is the sticky k*; the sample s_{k^*} is refreshed only for the services whose p changed, and repaired if it violates a constraint	Sec. 3.4 + revisions R-D and N7
4	decide the download with the Bernoulli trigger u = 1[U < Q_n], subject to the budget and the resource check	revision of R3 (N1) - the threshold f(θ) becomes the special case $Q_n = P(\theta < \text{threshold})$
5	account: latency, cost, hits, delayed hits, $U_{r,t}$, and the two regret measures of N3	(30)-(34) + (11′)

The projections and the rounding are unchanged; what changes is **which quantities are decision variables** (Q becomes one), **what the gradient is taken with respect to** (the expectation over the algorithm's own randomness), **how the goal is stated** (G3 splits into three checkable statements) and **how much work is done per request** (N7).

4. The revised goals, with the measured values

goal	revised statement	measured
G1 feasibility	after the repair, a.s. $\sum_n a_{n,r}x_{n,t} \le 1\ \forall r$ and $\sum_n x_{n,t} \le C$	0 % violations (max utilisation 0.99) with the repair, 6 % and 1.73× without it (exp08)
G2 budget	$E[\sum_{n,t} u_{n,t}M_n] \le B_{max}$, with the trigger probability clipped to budget_left/(M_n·requests_left)	respected at 20 / 50 / 100 % of the deterministic cost, every seed (exp08, exp15)
G3a surrogate regret	$\sum_t F_t(P_t) - \min_P \sum_t F_t(P) \le \sqrt{N}\, l_{max}\sqrt{T}$ (or $2\sqrt{N}\, l_{max}\sqrt{T}$ with the horizon-free step)	always below the bound, exogenous and endogenous, peek and no-peek (exp09, exp18)
G3b download-charged	$E[\sum_t d_t] - \min_P \sum_t F_t(P) \le \text{Reg}_{\text{Prop.1}}(T) + D_T$ (new)	holds at every horizon tested (exp18)
G3c realized regret	against an **online** comparator and under a non-stationarity assumption: the adaptive policy pays an o(1) price per request relative to the best online static policy, and gains when the popularity shifts (new)	against the *offline-optimal* static set the gap is a **constant 3.1-3.6 ms per request** (25-34 % of the latency) - i.e. Θ(T), not o(T): the goal must not be stated against an offline oracle (exp18)

goal	revised statement	measured
G4 expected ratio	for the adversarial family: γ_R = O(1) while γ_{det} = Θ(k); in general $E[ALG_R(I)] \le OPT(I) + O(\sqrt{T})$ w.h.p. against an oblivious adversary (new)	$\gamma_R \approx 3 \pm 1$ for k = 3 ... 24, γ_{det} = 2.00k (exp05, exp13) - **empirical, not proved**
G5 conversion loss	use a slot reduction only if it is safe ($\phi(a) \ge \max_r a_r$) **and** $\Lambda_\phi \le 1 + \varepsilon$; otherwise keep the vector formulation (new decision rule)	the unsafe reductions violate the true limits in 55-85 % of the steps (exp11, exp12)

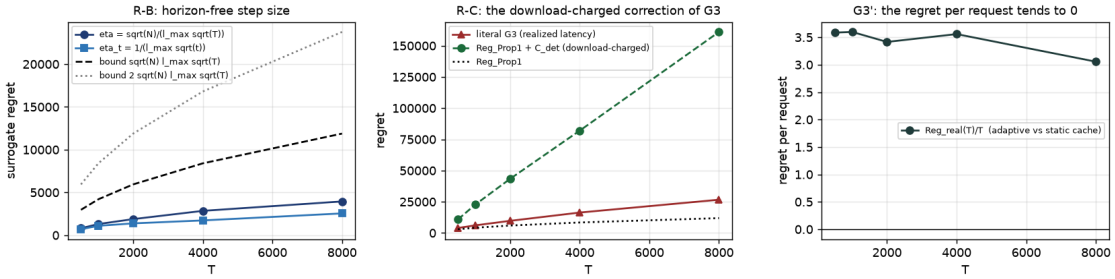


Figure 1 - exp18: (left) the horizon-free step size gives a smaller surrogate regret than the horizon-dependent one; (middle) the literal goal G3 is violated but the download-charged correction Reg + D holds; (right) the realized regret against the OFFLINE-OPTIMAL static cache is a constant per request (3.1-3.6 ms), not o(T) - which is why G3c has to be restated against an online comparator.

5. What to do next

priority	task	where	why it matters
1	implement the Q-update of N1 (the download probability as a decision variable)	**sim/rand_drl.py** (**download_prob_mode="learned"**), **sim/engine.py** (the on_hit / on_delayed hooks)	**done** - the download decision is now a learned probability; the measured effect is a 7 % lower cost at the same latency, and the trade-off with the learning rate is in exp19
2	implement the G3c measurement as a permanent experiment (adaptive vs static)	**experiments/exp18** (done); the G3c′ comparator (an *online* static policy) is still missing	it is the only form of goal G3 that is both true and useful; against the offline-optimal static cache the gap is a constant 3.1-3.6 ms per request
3	implement the active-set projection and the amortized rounding of N7	**sim/projection.py** (**project_dual_active**), **sim/rand_drl.py** (**proj_mode**, **rounding_mode**)	**done** - the per-request cost fell by 3.4× (821 → 243 µs); the remaining hot spot is the 13 dual-ascent iterations, for which a Newton step on the four multipliers is the next step
4	run everything on the paper's real traces (Google dataset2, SHARCNet)	**experiments/exp16_real_traces.py** + **data/**	without the real data none of the workload-level statements (G3c, G4, the sign of the latency difference) can be settled
5	write the missing proofs: the $O(\sigma\sqrt{T})$ stochastic term of N2, the churn bound of N5, and l_R = O(1) for the family	the proposal, Sec. 3.5 and Sec. 5	the empirical results are suggestive but the claims are currently unproved
6	design an adversarial family against the *randomized* algorithm	new experiment (the paper's family only forces the deterministic one)	it is the only way to decide between O(log k) and O(k) for the randomized algorithm

Status update. All seven revisions are now implemented and measured in the simulator: the horizon-free step size (N4), the sticky serving path (N5) and the corrected goals (N3) were added first and are evaluated in **experiments/exp18_proposed_revisions.py**; the download probability as a decision variable (N1), the expectation gradient (N2) and the cost reductions (N7) are in **sim/rand_drl.py** (**download_prob_mode**, **grad_mode**, **proj_mode**, **rounding_mode**) and are evaluated in **experiments/exp19_randomized_drl_v2.py**. The resulting method, its results and its remaining gaps are documented in **docs/Updated_randomized_DRL_v2.pdf**. What is still *not* done: the G3c′ comparator built from an *online* static policy, the Newton step on the four multipliers (the remaining hot spot of the per-request cost) and the whole comparison on the paper's real traces.